암호화폐 자동매매 플랫폼 개발 기획서

1. 프로젝트 개요 및 목표

1.1 프로젝트 개요

본 프로젝트는 암호화폐 자동매매를 시작으로, 향후 국내외 주식 투자까지 확장 가능한 통합 투자 플랫폼 개발을 목표로 합니다. 사용자가 직관적인 웹 인터페이스를 통해 투자 전략을 수립하고, 시스템이 자동으로 시장 데이터를 분석하여 매매를 실행하며, 실시간 모니터링과 성과 분석을 제공하는 종합적인 솔루션입니다.

1.2 핵심 목표

- **자동화된 투자 실행:** 사용자 정의 전략에 따른 자동 매매 시스템 구축
- **실시간 데이터 처리:** 시장 데이터, 뉴스, 차트 분석을 통한 의사결정 지원
- **확장 가능한 아키텍처:** 암호화폐에서 주식으로의 점진적 확장
- **사용자 친화적 대시보드:** 직관적인 모니터링 및 설정 관리 인터페이스
- **리스크 관리:** 손실 제한 및 포트폴리오 분산 투자 기능

1.3 타겟 사용자

- 개인 투자자 (초급~고급)
- 정량적 투자 전략을 선호하는 투자자
- 자동화된 투자 솔루션을 원하는 사용자
- 다양한 자산군에 분산투자를 희망하는 투자자

2. 기술 스택 분석 및 선택 기준

2.1 현재 기술 스택

- **OS:** Ubuntu (Docker 컨테이너)
- **Database:** MySQL / PostgreSQL (선택 필요)
- **Backend:** Python / NodeJS (선택 또는 병행 필요)

- **API Layer:** GraphQL (NodeJS)
- **Frontend:** NextJS
- **거래소 API:** Upbit, Coinone, Binance
- **증권사 API:** 키움증권, 한국투자증권, LS증권, 대신증권

2.2 기술 선택 기준

기술 영역	옵션	선택 기준	권장사항
Database	MySQL vs PostgreSQL	JSON 처리, 확장성, 복잡한 쿼리 지원	PostgreSQL (JSON 지원, 고급 기능)
Backend	Python vs NodeJS	데이터 분석, API 통합, 개발 속도	Python (데이터 분석) + NodeJS (API)
컨테이너	Docker	환경 일관성, 배포 편의성	Docker Compose 활용

3. 기획 단계 결정 사항 목록 (우선순위별)

3.1 최우선 결정 사항 (Phase 1)

P1-1. 비즈니스 모델 정의

- 수익 모델: 구독료 vs 거래 수수료 vs 성과 수수료
- 사용자 등급별 기능 차별화
- 무료 체험 기간 및 제한사항

고려사항: 초기 사용자 확보를 위한 프리미엄 전략 vs 수익성 확보

P1-2. 법적 요구사항 및 규제 준수

- 금융정보업 등록 필요성 검토
- 개인정보 보호법 준수 방안
- 암호화폐 관련 규제 대응
- 투자자 보호 조치 및 면책 조항

고려사항: 법무 검토 및 라이선스 취득 기간 고려

P1-3. 핵심 기능 범위 정의

- MVP 기능 선별 (최소 기능 제품)
- 암호화폐 거래소 우선순위 (Upbit → Binance → Coinone)
- 기본 투자 전략 유형 정의
- 리스크 관리 필수 기능

3.2 중요 결정 사항 (Phase 2)

P2-1. 사용자 인증 및 보안 체계

- 2FA (Two-Factor Authentication) 구현 방식
- API 키 관리 및 암호화 방식
- 사용자 권한 관리 체계
- 보안 감사 및 로깅 정책

P2-2. 데이터 수집 및 분석 전략

- 실시간 데이터 vs 배치 데이터 처리 비중
- 뉴스 데이터 소스 및 감정 분석 방식
- 기술적 지표 계산 라이브러리 선택
- 데이터 저장 및 백업 정책

3.3 일반 결정 사항 (Phase 3)

P3-1. 성능 최적화 방향

- 캐싱 전략 (Redis vs Memcached)
- 데이터베이스 인덱싱 정책
- API 요청 제한 및 최적화

- 프론트엔드 번들링 및 최적화

4. 디자인 단계 결정 사항 목록

4.1 UI/UX 디자인 방향

D1. 디자인 철학 및 컨셉

- 타겟 사용자별 인터페이스 복잡도 결정
- 색상 체계: 전문적 vs 친근한 이미지
- 정보 밀도: 대시보드 정보량 최적화
- 모바일 우선 vs 데스크톱 우선 설계

4.2 사용자 경험 설계

D2. 사용자 플로우 정의

- 온보딩 프로세스: 튜토리얼 vs 가이드 투어
- 전략 생성 워크플로우 단계별 설계
- 알림 및 경고 시스템 UX
- 에러 처리 및 사용자 피드백 방식

4.3 반응형 디자인 및 접근성

D3. 크로스 플랫폼 지원

- 브레이크포인트 정의 (모바일, 태블릿, 데스크톱)
- 터치 인터페이스 최적화
- 웹 접근성 지침 (WCAG) 준수 수준
- 다국어 지원 범위 (한국어, 영어)

4.4 데이터 시각화

D4. 차트 및 그래프 라이브러리

- 차트 라이브러리 선택: Chart.js vs D3.js vs TradingView
- 실시간 데이터 업데이트 방식
- 커스터마이징 가능한 대시보드 레이아웃
- 데이터 내보내기 형식 (PDF, Excel, CSV)

5. 개발 단계 결정 사항 목록

5.1 아키텍처 설계

DEV1. 시스템 아키텍처

- 마이크로서비스 vs 모노리스 아키텍처
- 메시지 큐 시스템: Redis Pub/Sub vs RabbitMQ
- 로드 밸런싱 및 고가용성 설계
- 백엔드 서비스 분리: 데이터 수집, 전략 실행, 사용자 관리

5.2 데이터베이스 설계

DEV2. 데이터 모델링

- 사용자 데이터 스키마 설계
- 거래 데이터 및 이력 관리 구조
- 실시간 데이터 vs 히스토리 데이터 분리
- 인덱싱 전략 및 쿼리 최적화

5.3 API 설계 및 통합

DEV3. 외부 API 통합

- 거래소 API 래퍼 클래스 설계

- API 요청 제한 및 에러 핸들링
- 웹소켓 연결 관리 및 재연결 로직
- API 키 보안 저장 및 관리

5.4 보안 및 인프라

DEV4. 보안 구현

- JWT 토큰 관리 및 갱신 전략
- HTTPS/SSL 인증서 관리
- 데이터 암호화 수준 (저장, 전송)
- 백업 및 재해 복구 계획

6. 각 결정사항에 대한 상세 설명과 고려사항

6.1 기술 스택 선택 상세

6.1.1 데이터베이스 선택: PostgreSQL 권장

PostgreSQL 선택 이유:

- JSON/JSONB 데이터 타입 지원으로 유연한 스키마 설계
- 복잡한 쿼리 및 분석 함수 지원
- 시계열 데이터 처리에 적합한 확장 기능
- 강력한 ACID 트랜잭션 지원

고려사항: MySQL 대비 초기 설정이 복잡하지만, 장기적으로 데이터 분석 및 확장성 측면에서 유리

6.1.2 백엔드 언어: Python + NodeJS 병행

역할 분담 제안:

- **Python:** 데이터 분석, 머신러닝, 투자 전략 계산 엔진
- **NodeJS:** API 서버, 실시간 데이터 처리, GraphQL 엔드포인트

고려사항: 두 언어 병행 시 개발 복잡도 증가, 하지만 각 언어의 장점을 최대화 가능

6.2 비즈니스 모델 고려사항

6.2.1 수익 모델 비교분석

모델	장점	단점	적용 시기
구독료	안정적 수익, 예측 가능	초기 사용자 확보 어려움	서비스 안정화 후
거래 수수료	사용량 비례, 진입 장벽 낮음	시장 변동성에 영향	초기 단계부터
성과 수수료	사용자와 이해관계 일치	수익 불안정성	고도화 단계

6.3 보안 고려사항

6.3.1 API 키 관리 전략

- **암호화:** AES-256 암호화로 데이터베이스 저장
- **권한 분리:** 읽기 전용 키와 거래 권한 키 분리
- **만료 관리:** 정기적인 키 갱신 및 권한 검증
- **감사 로그:** 모든 API 호출 기록 및 모니터링

6.3.2 거래 보안 체계

- **이중 확인:** 거래 실행 전 사용자 승인 단계
- **한도 설정:** 일일/월간 거래 한도 및 손실 제한
- **비정상 거래 탐지:** 패턴 분석을 통한 이상 거래 차단

7. 자동매매 전략 프로세스 체계화

7.1 전략 생명주기 개요

자동매매 시스템의 핵심은 지속적으로 개선되는 전략 관리 프로세스입니다. 전략의 생성부터 적용, 평가, 개선까지의 순환적 프로세스를 통해 시장 변화에 적응하고 수익성을 지속적으로 향상시킵니다.

> 전략 프로세스 플로우
>
> **매매 전략 생성 → 매매 전략 적용 → 성과 분석 및 평가 → 전략 최적화 → 전략 적용 → ...**

7.2 매매 전략 생성 단계

7.2.1 전략 연구 및 학습

기존 매매 전략 분석

- **기술적 분석 전략:** 이동평균, RSI, MACD, 볼린저밴드 등 지표 기반 전략
- **기본적 분석 전략:** 프로젝트 펀더멘털, 토크노믹스, 개발 진척도 기반 전략
- **감정 분석 전략:** 뉴스, SNS, 커뮤니티 감정 지수 기반 전략
- **차익거래 전략:** 거래소 간 가격 차이, 김치프리미엄 활용 전략
- **그리드 전략:** 구간별 매수/매도 자동 실행 전략
- **DCA 전략:** 달러 평균 비용 분할 매수 전략

전략별 특성 분석

전략 유형	장점	단점	적합 매매 주기	리스크 수준
기술적 분석	객관적 지표, 빠른 판단	시장 급변 시 한계	단기~중기	중간
기본적 분석	장기 트렌드 포착	단기 변동성 높음	중장기	낮음
감정 분석	시장 심리 반영	노이즈 많음	초단기~단기	높음
차익거래	안정적 수익	기회 제한적	초단타	낮음
그리드 전략	횡보장에서 유리	트렌드 장에서 불리	중기	중간

7.2.2 전략 설계 및 모델링

전략 매개변수 정의

- **진입 조건:** 매수 시점 결정 로직 및 임계값
- **청산 조건:** 매도 시점 결정 로직 (익절, 손절)
- **포지션 크기:** 총 자금 대비 매매 비중 결정

- **리스크 관리:** 최대 손실 한도, 연속 손실 제한
- **매매 주기:** 전략 실행 빈도 (실시간, 시간별, 일별 등)

백테스트 데이터 준비

- **가격 데이터:** OHLCV 데이터 (최소 1년, 권장 3년 이상)
- **거래량 데이터:** 시장 유동성 및 변동성 분석
- **뉴스 데이터:** 주요 이벤트, 공시, 업데이트 정보
- **소셜 데이터:** 트위터, 레딧, 텔레그램 감정 지수
- **온체인 데이터:** 거래량, 홀더 분포, 스테이킹 비율 등
- **거시경제 데이터:** 금리, 달러 지수, 주식 시장 상관관계

7.2.3 전략 검증 및 최적화

백테스트 시뮬레이션

- **기간별 테스트:** 불장, 곰장, 횡보장 구간별 성과 분석
- **매개변수 최적화:** 그리드 서치, 유전 알고리즘 활용
- **워크포워드 분석:** 시점별 점진적 검증
- **몬테카를로 시뮬레이션:** 다양한 시나리오 스트레스 테스트

성과 지표 평가

- **수익률:** 연환산 수익률, 알파, 베타
- **리스크 지표:** 최대 낙폭(MDD), 샤프 비율, 칼마 비율
- **거래 효율성:** 승률, 평균 수익/손실 비율
- **안정성:** 변동성, 드로다운 기간, 복구 시간

7.3 매매 전략 적용 단계

7.3.1 실시간 매매 시스템

초단타 매매 (High-Frequency Trading)

- **실행 주기:** 초~분 단위 매매 결정
- **데이터 소스:** 웹소켓 실시간 가격, 호가창 데이터
- **전략 유형:** 차익거래, 모멘텀, 평균 회귀
- **기술 요구사항:** 저지연 시스템, 높은 처리량
- **리스크 관리:** 실시간 포지션 모니터링, 즉시 손절

정기 매매 (Scheduled Trading)

- **일간 매매:** 매일 정해진 시간 매매 실행
  - 예시: 매일 오전 9시 시장 분석 → 10시 매수 → 오후 3시 매도
- **주간 매매:** 주간 패턴 기반 매매
  - 예시: 매주 화요일 DCA 매수, 금요일 익절 매도
- **이벤트 기반:** 특정 조건 충족 시 매매
  - 예시: 변동성 임계값 초과 시, 뉴스 이벤트 발생 시

7.3.2 포트폴리오 관리

자금 배분 전략

- **고정 비중:** 각 코인별 고정 투자 비율 유지
- **동적 리밸런싱:** 시장 상황에 따른 비중 조정
- **켈리 기준:** 수학적 최적 포지션 크기 계산
- **리스크 패리티:** 리스크 기여도 기반 배분

7.4 성과 분석 및 평가 단계

7.4.1 실시간 모니터링

핵심 성과 지표 (KPI) 추적

- **일간 P&L:** 일별 손익 현황
- **누적 수익률:** 기간별 누적 수익률 추이
- **현재 MDD:** 현재 최대 낙폭 수준
- **포지션 현황:** 보유 자산 및 비중
- **거래 빈도:** 일간 거래 횟수 및 거래량

7.4.2 정기 성과 리뷰

주간/월간 분석 리포트

- **수익률 분석:** 절대 수익률 vs 벤치마크 비교
- **리스크 분석:** 변동성, 드로다운 패턴 분석
- **거래 분석:** 승률, 평균 보유 기간, 거래 비용
- **전략별 기여도:** 각 전략의 수익 기여도 분석

7.5 전략 최적화 및 개선 단계

7.5.1 문제점 진단

성과 저하 원인 분석

- **시장 환경 변화:** 거시경제, 규제, 기술적 변화 영향
- **전략 오버피팅:** 과거 데이터에 과도하게 최적화
- **실행 차이:** 백테스트 vs 실제 거래 결과 괴리
- **매개변수 노화:** 시장 변화로 인한 최적 매개변수 변화

7.5.2 전략 업데이트 프로세스

개선 방법론

- **매개변수 재최적화:** 최근 데이터 기반 매개변수 조정
- **신규 지표 추가:** 시장 변화를 반영하는 새로운 지표 도입
- **앙상블 전략:** 여러 전략의 조합을 통한 리스크 분산

- **적응형 전략:** 시장 상황에 따라 자동 조정되는 전략

업데이트 검증 절차

1. **가설 수립:** 개선 방향 및 예상 효과 정의
2. **백테스트 검증:** 수정된 전략의 과거 성과 검증
3. **페이퍼 트레이딩:** 실제 시장에서 모의 거래 실행
4. **점진적 적용:** 소액으로 시작하여 단계적 확대
5. **A/B 테스트:** 기존 전략과 병행 운용하여 비교

7.6 전략 관리 시스템 요구사항

7.6.1 기술적 구현 요소

전략 엔진 아키텍처

- **전략 저장소:** 전략 코드 버전 관리 및 배포
- **백테스트 엔진:** 대용량 데이터 처리 및 병렬 계산
- **실행 엔진:** 실시간 신호 생성 및 주문 실행
- **모니터링 시스템:** 전략 성과 추적 및 알림
- **리스크 관리:** 실시간 리스크 모니터링 및 제어

7.6.2 사용자 인터페이스

전략 관리 대시보드

- **전략 빌더:** 드래그앤드롭 방식의 전략 생성 도구
- **백테스트 도구:** 시각적 백테스트 결과 및 분석
- **실시간 모니터링:** 전략별 실시간 성과 대시보드
- **알림 시스템:** 중요 이벤트 및 임계값 도달 알림
- **보고서 생성:** 자동화된 성과 리포트 생성

8. 향후 PRD, TRD, Task TODO 리스트 작성을 위한 준비사항

8.1 PRD (Product Requirements Document) 준비사항

8.1.1 필수 선행 결정사항

- 타겟 사용자 페르소나 구체화
- 핵심 기능별 우선순위 확정
- 경쟁사 분석 및 차별화 포인트 정의
- 성공 지표 (KPI) 설정

8.1.2 수집 필요 정보

- 거래소별 API 제한사항 및 비용 구조
- 유사 서비스 가격 정책 조사
- 법적 요구사항 상세 조사 결과
- 초기 사용자 테스트 결과 및 피드백

8.2 TRD (Technical Requirements Document) 준비사항

8.2.1 기술적 요구사항 정의

- 시스템 성능 요구사항 (응답시간, 처리량)
- 확장성 요구사항 (동시 사용자 수, 데이터 용량)
- 보안 요구사항 상세 명세
- 외부 시스템 연동 요구사항

8.2.2 인프라 설계 요소

- 클라우드 서비스 선택 (AWS, GCP, Azure)
- 모니터링 및 로깅 시스템 설계
- CI/CD 파이프라인 구성
- 테스트 환경 및 전략

8.3 Task TODO 리스트 준비사항

8.3.1 개발 단계별 작업 분류

- **Phase 1:** 기반 인프라 및 핵심 기능 개발

- **Phase 2:** 고급 기능 및 사용자 경험 개선
- **Phase 3:** 확장 기능 및 최적화

8.3.2 작업 추정 기준

- 개발자 역량 및 경험 수준 평가
- 외부 의존성 (API 승인, 라이선스 등) 고려
- 테스트 및 QA 시간 포함
- 리스크 요소별 버퍼 시간 설정

8.4 다음 단계 권장사항

1. **결정사항 우선순위 검토:** 상기 결정사항들을 팀 내에서 검토하고 우선순위 조정
2. **프로토타입 개발:** 핵심 기능 중심의 간단한 프로토타입 제작
3. **사용자 조사:** 타겟 사용자 인터뷰 및 니즈 분석
4. **기술 검증:** 선택한 기술 스택으로 POC (Proof of Concept) 구현
5. **법무 검토:** 금융 서비스 관련 법적 요구사항 전문가 검토

본 문서는 암호화폐 자동매매 플랫폼 개발을 위한 초기 기획 단계 문서로, 구체적인 개발 진행 전 모든 이해관계자의 검토와 승인이 필요합니다.